

COP701 Assignment 3: Mini Web Browser Report

Shuvam Chakraborty (2024MCS2004) & Saurabh Joshi (2024MCS2003)

November 16, 2024

1 Introduction

The objective of this assignment was to create a multi-threaded, multi-process mini web browser that can fetch, parse, and render simple HTML web pages. The browser supports the parallel handling of different tabs using separate processes for each of the core tasks: fetching HTML pages, parsing them into a DOM (Document Object Model), and rendering the parsed content.

The following components were implemented:

- Fetch HTML pages using HTTP requests and cache them locally.
- Parse HTML pages into a DOM structure.
- Render the DOM structure using Qt Widgets.
- Implement multi-threading and multi-processing to enhance performance and responsiveness.

The project was developed in C++ and utilized various technologies and libraries including `libcurl`, `flex`, `bison`, and `Qt Widgets`. Multi-threading was used within the processes to handle concurrent tasks.

2 System Architecture and Design

The system is designed with three main processes that communicate with each other in a producer-consumer pattern:

- **Network Fetcher Process:** Fetches HTML pages using `libcurl` and caches them locally.
- **Parser Process:** Converts the cached HTML pages into a DOM structure using a custom parser built with `flex` and `bison`.
- **Renderer Process:** Reads the DOM structure from the file and renders the content using `Qt Widgets`.

2.1 Communication and Synchronization

The communication between these processes follows the producer-consumer model:

- The **Network Fetcher** acts as the producer, fetching HTML pages from a local server.
- The **Parser** acts as the consumer, receiving the fetched content and parsing it into a DOM structure.
- The **Renderer** consumes the DOM and renders it on the screen.

Multi-threading is used within each process to handle multiple tasks concurrently, enhancing performance.

3 Implementation Details

3.1 Network Fetching

Functionality: The network fetcher process fetches HTML pages from a local server using `libcurl`. The server is simulated with the command `python3 -m http.server`. Once the pages are fetched, they are cached in the local directory to avoid repeated network requests and improve performance.

- **Technology Used:** `libcurl`, `python3 -m http.server`.

3.2 HTML Parsing to DOM

Functionality: A custom parser is built using `flex` (for lexical analysis) and `bison` (for parsing). The parser processes the fetched HTML pages and converts them into a DOM/AST (Abstract Syntax Tree). The parser supports a restricted set of HTML tags, including `html`, `head`, `title`, `body`, `p`, `h1`, `img`, `a`, etc.

- **Technology Used:** `flex`, `bison`.

3.3 Rendering Engine

Functionality: The renderer process reads the DOM structure and renders the corresponding content on the screen using `Qt Widgets`. Text is drawn using basic widget elements in `Qt`.

- **Technology Used:** `Qt Widgets`.

3.4 Multi-Processing and Multi-Threading

Implementation: The project implements multi-processing to separate the tasks of fetching, parsing, and rendering. Each process runs in its own thread, with communication between processes facilitated by shared data structures. Multi-threading within each process handles concurrent tasks to improve performance.

- **Libraries Used:** `std::thread`.

4 Challenges and Solutions

4.1 Challenge: Synchronizing Data Flow Between Processes

Synchronizing the flow of data between processes was a challenge, as the fetcher, parser, and renderer processes had to operate in a strict sequence. The data produced by the fetcher needed to be consumed by the parser, which in turn needed to pass the DOM to the renderer.

Solution: This was solved by using multi-threading to handle concurrent tasks within each process. Synchronization between the processes was handled via shared memory and thread communication.

4.2 Challenge: Developing a Custom HTML Parser

Another challenge was building a custom HTML parser from scratch using `flex` and `bison`. Ensuring that the parser was both efficient and compliant with the assignment's restricted tag set was tricky.

Solution: By using `flex` for lexical analysis and `bison` for parsing, we created an efficient parser that handles the specified HTML tags. We also incorporated error handling to ensure robustness.

5 Testing and Validation

Test Environment: HTML pages were served locally using `python3 -m http.server`. The HTML dataset was generated using the provided `gen.html.py` script. Various test cases were designed to validate the browser's ability to correctly parse and render different HTML tags and structures.

Test Cases: The browser was tested with the following:

- Simple HTML pages containing basic tags such as `html`, `head`, `body`, and `title`.

- Complex HTML pages with images (`img`) and hyperlinks (`a`).
- Pages with various text elements (`p`, `h1`) to verify correct text rendering.

6 Bonus Features

6.1 Doxygen Documentation

As part of the project, we have created a Doxygen documentation that provides an in-depth overview of the codebase. This includes the description of classes, methods, and their interactions. The documentation is available in the `documentation` folder and can be accessed in HTML format.

6.2 History Tracking

A history tracking feature was added, where the browser logs visited URLs in a `history.txt` file. This allows users to track previously visited pages, a useful feature for future enhancements, such as session restoration.

7 Conclusion

The mini web browser project successfully meets the requirements of the assignment. Key features include:

- Multi-process architecture for fetching, parsing, and rendering HTML content.
- Custom HTML parser using `flex` and `bison`.
- Rendering using `Qt Widgets` with support for text and images.
- Multi-threading within processes for enhanced performance.
- Bonus features: Doxygen documentation and history tracking.

Future improvements could include enhancing the HTML parser to support more complex HTML structures and adding features such as session restoration and advanced error handling.